

Creating the Creature, With Phidgets!

Phidgets are a great, simple, way to connect your digital application to the real world. The phidgets SDK offers a straightforward approach to connecting, monitoring, and adjusting a variety of sensors.

These guides will walk you through creating a single html file that connects to a Phidget server, monitors a Touch Sensor, monitors a Slider, and finally controls an LED in response to user input. This guide will focus on the first of those, connecting to a Phidget server and monitoring a Touch Sensor.

What we're building

A single HTML file, opened directly in a browser, that:

1. Connects to a Phidget Network Server over the network.
2. Listens to a digital input (a touch sensor) plugged into a Phidget hub.
3. Shows a raw JSON dump of whatever the Phidget library is telling us, for debugging.
4. Shows a button that changes color as you press the button.

Everything lives in one `.html` file — no build tools, no `npm install`, no server needed on your end. You just double-click the file (or serve it) and it runs.

Step 1 — The skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Phidget Monitor</title>
  <script src="https://unpkg.com/phidget22/browser/phidget22.js"></script>
</head>
</html>
```

This is boilerplate every HTML page needs, plus one important line: the `<script src="...">` tag. This pulls in Phidget's official browser SDK from a public CDN (a server that just hosts files). Once this line runs, a global object called `phidget22` becomes available to the rest of our page — that's the library we'll use to talk to the hardware.

Step 2 — The visual layout

Next we'll start building out our web page. The body has three sections:

1. **A connection form** — text inputs for the server address/port and a Connect/Disconnect button.
2. **A button visual** — an empty `<div>` that we'll color and label from JavaScript.
3. **A raw state panel** — another `<div>` that will show JSON text.

```
<body>
  <h1>Phidget Monitor</h1>

  <!-- The connection form -->
  <section>
    <form id="connectionForm">
```

```

    <label>
      Server address
      <input id="serverAddress" type="text" value="localhost" />
    </label>
    <label>
      Port
      <input id="serverPort" type="number" value="8989" style="width: 80px;" />
    </label>
    <button id="connectButton" type="submit">Connect</button>
    <button id="disconnectButton" type="button" disabled>Disconnect</button>
  </form>
  <div id="statusLine">
    Status: <span id="statusText" class="disconnected">Disconnected</span>
  </div>
</section>

<!-- The button visual -->
<section>
  <h2>Button State</h2>
  <div id="buttonVisual" class="neutral">NEUTRAL</div>
</section>

<!-- The raw state panel -->
<section>
  <h2>Raw Phidget State</h2>
  <div id="rawStatePanel">Waiting for connection...</div>
</section>
</body>

```

Each element that JavaScript needs to control later has an `id` attribute (e.g. `id="buttonVisual"`). Think of an `id` as a unique name tag — JavaScript uses `document.getElementById("buttonVisual")` to grab that exact element and change it. Nothing shows up dynamically until JavaScript reaches in and modifies it.

Step 3 — Grabbing references to the elements

Next we start building out the script. This will make our page come alive and make our page react to user input and phidgets.

To start, we need to get a reference to all of the DOM elements we'll be working with in our script.

```

<body>
  ...
  <script>
    const connectionForm = document.getElementById("connectionForm");
    const serverAddressInput = document.getElementById("serverAddress");
    const serverPortInput = document.getElementById("serverPort");
    const connectButton = document.getElementById("connectButton");
    const disconnectButton = document.getElementById("disconnectButton");
    const statusText = document.getElementById("statusText");
    const buttonVisual = document.getElementById("buttonVisual");
    const rawStatePanel = document.getElementById("rawStatePanel");
  </script>

```

```
</script>
</body>
```

This is done once, up front, and stored in `const` (constant — the variable itself never gets reassigned to point at something else) variables. Every function below reuses these same references instead of re-querying the page each time.

Just below these, let's add a couple more variables we'll use later on.

```
let connection;
let digitalInput;
```

These two variables will hold the Phidget connection, and digitalInput reference. We use `let` here since we plan to update these in response to the user clicking the "connect" and "disconnect" buttons.

Step 4 — Helper functions that only touch the display

Add the following in the script tag, after the section where we look up the elements.

```
/// -----
/// Helper functions
/// -----

function updateRawStatePanel(stateObject) {
  rawStatePanel.textContent = JSON.stringify(stateObject, null, 2);
}

function setConnectedUI(isConnected) {
  statusText.textContent = isConnected ? "Connected" : "Disconnected";
  statusText.className = isConnected ? "connected" : "disconnected";
  connectButton.disabled = isConnected;
  disconnectButton.disabled = !isConnected;
  serverAddressInput.disabled = isConnected;
  serverPortInput.disabled = isConnected;
}

function setButtonVisual(isPressed) {
  buttonVisual.classList.toggle("pressed", isPressed);
  buttonVisual.classList.toggle("neutral", !isPressed);
  buttonVisual.textContent = isPressed ? "PRESSED" : "NEUTRAL";
}
```

These three functions have one job each and don't talk to Phidget hardware at all — they just take a value and update what's on screen.

- `updateRawStatePanel` takes any JavaScript object and turns it into readable, indented JSON text with `JSON.stringify(..., null, 2)`. The `2` means "indent with 2 spaces," which is what makes the raw state panel readable instead of one long line.
- `setConnectedUI` flips the Connect/Disconnect buttons' enabled state and updates the status label. Disabling the form fields while connected prevents someone from editing the address mid-connection and confusing the app.

- `setButtonVisual` toggles CSS classes (`pressed` / `neutral`) rather than manually rewriting styles. The colors themselves live in the `<style>` block — this function only decides *which* state applies.

Step 5 — Connecting to the server

Finally! Let's connect to the Phidget server.

```
/// -----
/// Phidget Controls
/// -----

async function connectToPhidgetServer(address, port) {
  connection = new phidget22.NetworkConnection({
    hostname: address,
    port: port,
    onError: (code, message) => updateRawStatePanel({ connectionError: { code, message } }),
    onConnect: () => updateRawStatePanel({ status: "Connected" }),
    onDisconnect: () => {
      setConnectedUI(false);
      updateRawStatePanel({ status: "Disconnected from server" })
    }
  });

  await connection.connect();
}
```

`async function` marks this function as one that does work which takes time (like reaching across the network) without freezing the page while it waits. Inside it, `await connection.connect()` means "pause this function here until the connection either succeeds or fails, but let the rest of the page keep working in the meantime." This is the standard way JavaScript handles anything that isn't instant — network calls, file access, timers.

`NetworkConnection` is a Phidget22 object representing the link to the Phidget Network Server. We hand it three callback functions:

- `onError` — fires if something goes wrong at the connection level.
- `onConnect` — fires once the connection is live.
- `onDisconnect` — fires if the server drops us, and importantly calls `setConnectedUI(false)` so the page reflects reality instead of pretending we're still connected.

Once we are connected, we still need to create connections for the Individual inputs that we want to monitor.

```
/// -----
/// Phidget Controls
/// -----

async function connectToPhidgetServer(address, port) {
  ...
  await connection.connect();

  await openDigitalInput();

  setConnectedUI(true);
}
```

```

}

async function openDigitalInput() {
  digitalInput = new phidget22.DigitalInput();

  digitalInput.isHubPortDevice = true;
  digitalInput.hubPort = 0;
}

```

`DigitalInput` represents one specific type of Phidget channel — a simple on/off sensor, which is exactly what a touch sensor is. Setting `isHubPortDevice = true` and `hubPort = 0` tells the library: "don't search for just any digital input on the network — target the device plugged directly into port 0 of the hub." This matters once more than one Phidget is on the network; without pinning the port, the library would grab whichever digital input it found first, which may not be the button you're pressing.

```

async function openDigitalInput() {
  ...

  digitalInput.onAttach = () => {
    updateRawStatePanel({
      status: "Attached",
      deviceName: digitalInput.deviceName,
      serialNumber: digitalInput.deviceSerialNumber,
      channel: digitalInput.channel,
      state: digitalInput.state
    });
    setButtonVisual(digitalInput.state === true);
  };

  digitalInput.onDetach = () => {
    updateRawStatePanel({ status: "Device detached" });
    setButtonVisual(false);
  };

  digitalInput.onStateChange = (state) => {
    updateRawStatePanel({
      status: "State change",
      deviceName: digitalInput.deviceName,
      serialNumber: digitalInput.deviceSerialNumber,
      channel: digitalInput.channel,
      state: state,
      timestamp: new Date().toISOString()
    });
    setButtonVisual(!state);
  };

  await digitalInput.open(5000);
}

```

Three more callbacks, this time on the channel itself:

- `onAttach` fires once the library has actually found and linked to the physical device. This is where we read its initial state — the button might already be held down when the page loads.
- `onDetach` fires if the device is unplugged or the connection is lost. We reset the visual to neutral rather than leaving it showing a stale "pressed" state.
- `onStateChange` is the one that matters for live interaction — it fires every time the button is pressed or released. `setButtonVisual(!state)` inverts the raw hardware value before displaying it, because this particular sensor reports `false` when physically pressed and `true` when released. The raw state panel below it always shows the *unmodified* value so you can tell which way your hardware actually behaves.

`digitalInput.open(5000)` starts the search for the matching device and waits up to 5000 milliseconds (5 seconds) for it to attach before giving up. `setConnectedUI(true)` only runs after that succeeds, so the UI can't claim "Connected" before there's actually a working channel.

At this point, you can open your html file in the browser, and type `connectToPhidgetServer('localhost', 8989)` in the console. Your UI will update as it connects to the server and begins receiving signals from the sensor. Although we have a few more things to finish.

Step 6 — Disconnecting cleanly

```
async function disconnectFromPhidgetServer() {
  if (digitalInput) {
    try {
      await digitalInput.close();
    } catch (error) {
      console.warn(error);
      // Channel may already be closed — nothing to do here.
    }
    digitalInput = null;
  }
  setButtonVisual(false);
  setConnectedUI(false);
  updateRawStatePanel({ status: "Disconnected" });
}
```

The `try { ... } catch (error) { ... }` block means "attempt this, and if it throws an error, don't crash the page — just note it and move on." Closing an already-closed channel is one of those situations, so we swallow that specific failure rather than letting it stop the cleanup. Setting `digitalInput = null` afterward signals "there's currently no active channel," which other code can check.

Step 7 — Wiring up the buttons

```
/// -----
/// Event Listeners
/// -----

connectionForm.addEventListener("submit", async (event) => {
  event.preventDefault();
  const address = serverAddressInput.value.trim();
  const port = parseInt(serverPortInput.value, 10);
```

```

try {
  updateRawStatePanel({ status: "Connecting..." });
  await connectToPhidgetServer(address, port);
}
catch (error) {
  console.error(error);

  updateRawStatePanel({ connectionError: String(error) });
  setConnectedUI(false);
}
});

disconnectButton.addEventListener("click", disconnectFromPhidgetServer);

```

`addEventListener` is how JavaScript reacts to things a user does — a form submission, a button click. `event.preventDefault()` stops the browser's default behavior of reloading the page when a form is submitted, which is what would normally happen and would blow away our connection.

Trying it yourself

1. Make sure a Phidget Network Server is running and reachable, with a digital input device (a pushbutton) plugged into hub port 0.
2. Open `phidget-button.html` in a browser.
3. Enter the server's address and port (default `localhost:8989`) and click Connect.
4. Watch the raw state panel for the "Attached" message, then press the physical button — the circle and the raw state panel should update together.

If the circle doesn't match physical reality (pressed shows as neutral or vice versa), that's the inversion in `onStateChange` — flip the `!state` to `state` and reload.

Step 8 - Pizzazz!

Now our web page is functional. It connects to the Phidget Server and reacts as the user touches the sensor. But it could look way better.

Finally, let's build out our style sheet to make our page look a bit more modern.

In the head section of our page, add the following:

```

<head>
  ...
  <style>
    body {
      max-width: 720px;
      margin: 40px auto;
      padding: 0 16px;
      background: #f5f5f5;
      color: #1a1a1a;
    }

    h1 {

```

```
    font-size: 1.4rem;
}

/* Connection form */
#connectionForm {
    display: flex;
    gap: 8px;
    align-items: center;
    margin-bottom: 16px;
    flex-wrap: wrap;
}

#connectionForm input {
    padding: 6px 8px;
    font-size: 1rem;
}

#connectionForm button {
    padding: 6px 14px;
    font-size: 1rem;
    cursor: pointer;
}

#statusLine {
    margin-bottom: 24px;
    font-size: 0.95rem;
}

#statusLine .connected {
    color: #1a7f37;
    font-weight: bold;
}

#statusLine .disconnected {
    color: #b42318;
    font-weight: bold;
}

/* Button visual */
#buttonVisual {
    width: 180px;
    height: 180px;
    border-radius: 50%;
    margin: 32px auto;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.2rem;
    font-weight: bold;
    color: white;
    background: #8a8f98;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
}
```

```

    transition: background-color 0.1s ease;
    user-select: none;
}

#buttonVisual.pressed {
    background: #d3322b;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.3) inset;
}

#buttonVisual.neutral {
    background: #3a9d4f;
}

/* Raw state panel */
#rawStatePanel {
    background: #10131a;
    color: #b6ffb6;
    padding: 16px;
    border-radius: 8px;
    font-family: "Courier New", monospace;
    font-size: 0.85rem;
    white-space: pre-wrap;
    word-break: break-all;
    max-height: 300px;
    overflow-y: auto;
}

section {
    margin-bottom: 28px;
}

label {
    font-size: 0.9rem;
}
</style>
...
</head>

```

That's it! You have successfully connected your web page to physical hardware. In the next example we'll swap the sensor and UI to connect to a different type of sensor.

Bonus Round

Instead of just changing colors on the button as the user presses the sensor, try different ways of representing the state. Show / hide images, add a css animation, or something do something completely wild!